LAB 4

Connect Four
Updated Source Code

Code listing for the AI-assisted improvement submission

Included files: board.py, ai.py, game.py, main.py, and test_board.py.

Important: This PDF is a readable code listing. Keep and submit the original .py files too so the instructor can run the program.

board.py

```
ROWS, COLS = 6, 7


class Board:
    def __init__(self):
        self.grid = [["."] * COLS for _ in range(ROWS)]

    def drop(self, col, token):
        if not isinstance(col, int) or not 0 <= col < COLS:
            return None

        for r in range(ROWS - 1, -1, -1):
            if self.grid[r][col] == ".":
                self.grid[r][col] = token
                return r

        return None

    def full(self):
        return all(self.grid[0][c] != "." for c in range(COLS))

    def winner(self, token):
        directions = [
            (0, 1),   # Horizontal
            (1, 0),   # Vertical
            (1, 1),   # Diagonal down-right
            (1, -1),  # Diagonal down-left
        ]

        for r in range(ROWS):
            for c in range(COLS):
                if self.grid[r][c] != token:
                    continue

                for dr, dc in directions:
                    if all(
                        0 <= r + i * dr < ROWS
                        and 0 <= c + i * dc < COLS
                        and self.grid[r + i * dr][c + i * dc] == token
                        for i in range(4)
                    ):
                        return True

        return False

    def print(self):
        print("\n  " + " ".join(str(i + 1) for i in range(COLS)))
        for row in self.grid:
            print("  " + " ".join(row))
```

ai.py

```
class AI:
    def choose_column(self, board, me="O", opponent="X"):
        legal = [
            c for c in range(7)
            if board.grid[0][c] == "."
        ]

        if not legal:
            return None

        # First: take an immediate winning move.
        for col in legal:
            row = board.drop(col, me)

            if row is not None:
                wins = board.winner(me)
                board.grid[row][col] = "."

                if wins:
                    return col

        # Second: block an immediate opponent win.
        for col in legal:
            row = board.drop(col, opponent)

            if row is not None:
                opponent_wins = board.winner(opponent)
                board.grid[row][col] = "."

                if opponent_wins:
                    return col
```

```
        # Otherwise: prefer the centre, then nearby columns.
        return min(legal, key=lambda c: abs(c - 3))
```

game.py

```
from board import Board
from ai import AI


class Game:
    def __init__(self):
        self.board = Board()
        self.ai = AI()
        self.turn = "X"

    def run(self):
        print("Connect Four — you are X.")

        while True:
            self.board.print()

            if self.turn == "X":
                raw = input("Column (1-7), or q: ").strip().lower()

                if raw == "q":
                    print("Game ended.")
                    return

                try:
                    col = int(raw) - 1
                except ValueError:
                    print("Enter a column number from 1 to 7.")
                    continue

                if not 0 <= col < 7:
                    print("Choose a column from 1 to 7.")
                    continue

            else:
                col = self.ai.choose_column(self.board)

            if col is None:
                self.board.print()
                if self.board.full():
                    print("Draw.")
                else:
                    print("AI could not find a legal move. Game ended.")
                return

            row = self.board.drop(col, self.turn)

            if row is None:
                if self.turn == "O":
                    print("AI selected an unavailable column. Game ended.")
                    return

                print("Column full. Choose another column.")
                continue

            # Feedback occurs once, after a successful placement.
            print(f"{self.turn} placed a disc in column {col + 1}.")

            # Check win before asking for another turn.
            if self.board.winner(self.turn):
                self.board.print()
                print(self.turn, "wins!")
                return

            if self.board.full():
                self.board.print()
                print("Draw.")
                return

            self.turn = "O" if self.turn == "X" else "X"
```

main.py

```
from game import Game

if __name__ == "__main__":
    Game().run()
```

test_board.py

```
from board import Board
from ai import AI


def test_board_win(name, positions, expected):
    board = Board()

    for r, c in positions:
        board.grid[r][c] = "X"

    result = board.winner("X")
    assert result == expected, (
        f"{name}: expected {expected}, got {result}"
    )
    print(f"PASS: {name}")


def test_ai():
    ai = AI()

    # AI should win by placing O in column 4.
    board = Board()
    for c in range(3):
        board.grid[5][c] = "O"

    before = [row[:] for row in board.grid]
    move = ai.choose_column(board)
    assert move == 3, f"AI winning move expected 4, got {move + 1}"
    assert board.grid == before, "AI changed the board while analysing"
    print("PASS: AI takes immediate winning move")

    # AI should block X from winning in column 4.
    board = Board()
    for c in range(3):
        board.grid[5][c] = "X"

    before = [row[:] for row in board.grid]
    move = ai.choose_column(board)
    assert move == 3, f"AI blocking move expected 4, got {move + 1}"
    assert board.grid == before, "AI changed the board while analysing"
    print("PASS: AI blocks immediate player win")

    # A full board must not produce a move.
    board = Board()
    for r in range(6):
        for c in range(7):
            board.grid[r][c] = "X"

    assert ai.choose_column(board) is None
    print("PASS: AI handles a board with no legal moves")


def main():
    test_board_win("Horizontal win", [(5, 0), (5, 1), (5, 2), (5, 3)], True)
    test_board_win("Vertical win", [(2, 0), (3, 0), (4, 0), (5, 0)], True)
    test_board_win("Diagonal down-right win", [(1, 0), (2, 1), (3, 2), (4, 3)], True)
    test_board_win("Diagonal down-left win", [(1, 3), (2, 2), (3, 1), (4, 0)], True)
    test_board_win("Three in a row is not a win", [(5, 0), (5, 1), (5, 2)], False)

    board = Board()
    for r in range(6):
        board.grid[r][0] = "X"

    assert board.drop(0, "O") is None
    print("PASS: Full column rejects another disc")

    assert board.drop(-1, "O") is None
    assert board.drop(7, "O") is None
    print("PASS: Out-of-range columns are rejected")

    test_ai()
    print("\nAll automated tests passed!")


if __name__ == "__main__":
    main()
```